

Cheat Sheet: Python Data Structures Part-2

사전

패키지/메서드	설명	코드 예제
사전 생성	사전은 키-값 쌍의 컬렉션을 나타내는 내장 데이터 유형입니다. 사전은 중괄호 {}로 둘러싸여 있습니다.	예제: dict_name = {} #빈 사전을 생성합니다. person = { "name": "John", "age": 30, "city": "New York"}
값 접근	사전에서 값에 접근하려면 해당하는 키를 사용합니다.	구문: Value = dict_name["key_name"] 예제: name = person["name"] age = person["age"]
추가 또는 수정	사전에 새로운 키-값 쌍을 삽입합니다. 키가 이미 존재하면 값이 업데이트되고, 그렇지 않으면 새로운 항목이 생성됩니다.	구문: dict_name[key] = value 예제: person["Country"] = "USA" # 새로운 항목이 생성됩니다. person["city"] = "Chicago" # 동일한 키의 기존 값을 업데이트합니다.
del	지정된 키-값 쌍을 사전에서 제거합니다. 키가 존재하지 않으면 <code>KeyError</code> 가 발생합니다.	구문: del dict_name[key]

		예제: <pre>del person["Country"]</pre>
update()	update() 메서드는 제공된 사전을 기존 사전에 병합하여 키-값 쌍을 추가하거나 업데이트합니다.	구문: <pre>dict_name.update({key: value})</pre> 예제: <pre>person.update({"Profession": "Doctor"})</pre>
clear()	clear() 메서드는 사전을 비워 모든 키-값 쌍을 제거합니다. 이 작업 후에도 사전은 여전히 접근 가능하며 추가로 사용할 수 있습니다.	구문: <pre>dict_name.clear()</pre> 예제: <pre>grades.clear()</pre>
키 존재 여부	in 키워드를 사용하여 사전에서 키의 존재 여부를 확인할 수 있습니다.	예제: <pre>if "name" in person: print("사전에 이름이 존재합니다.")</pre>
copy()	사전의 얕은 복사본을 생성합니다. 새로운 사전은 원본과 동일한 키-값 쌍을 포함하지만 메모리에서 별개의 객체로 유지됩니다.	구문: <pre>new_dict = dict_name.copy()</pre>

		예제: <pre>new_person = person.copy() new_person = dict(person) # 사전의 복사를 생성하는 또 다른 방법</pre>
keys()	사전의 모든 키를 검색하여 리스트로 변환합니다. 리스트 메서드를 사용하여 키를 반복하거나 처리하는 데 유용합니다.	구문: <pre>keys_list = list(dict_name.keys())</pre> 예제: <pre>person_keys = list(person.keys())</pre>
values()	사전의 모든 값을 추출하여 리스트로 변환합니다. 이 리스트는 추가 처리나 분석에 사용할 수 있습니다.	구문: <pre>values_list = list(dict_name.values())</pre> 예제: <pre>person_values = list(person.values())</pre>
items()	모든 키-값 쌍을 튜플로 검색하여 튜플 리스트로 변환합니다. 각 튜플은 키와 해당 값을 포함합니다.	구문: <pre>items_list = list(dict_name.items())</pre> 예제: <pre>info = list(person.items())</pre>

집합

패키지/메서드	설명	코드 예제
add()	요소는 `add()` 메서드를 사용하여 집합에 추가할 수 있습니다. 중복은 자동으로 제거되며, 집합은 고유한 값만 저장합니다.	구문: <pre>set_name.add(element)</pre> 예제: <pre>fruits.add("mango")</pre>
clear()	`clear()` 메서드는 집합의 모든 요소를 제거하여 빈 집합을 만듭니다. 집합을 제자리에서 업데이트합니다.	구문: <pre>set_name.clear()</pre> 예제: <pre>fruits.clear()</pre>
copy()	`copy()` 메서드는 집합의 얇은 복사본을 생성합니다. 복사본에 대한 수정은 원본 집합에 영향을 미치지 않습니다.	구문: <pre>new_set = set_name.copy()</pre> 예제: <pre>new_fruits = fruits.copy()</pre>
집합 정의	집합은 고유한 요소의 순서가 없는 컬렉션입니다. 집합은 중괄호 `{}` 로 둘러싸여 있습니다. 고유한 값을 저장하고 집합 연산을 수행하는 데 유용합니다.	예제: <pre>empty_set = set() #빈 집합 생성 fruits = {"apple", "banana", "orange"}</pre>

discard()	<code>`discard()`</code> 메서드를 사용하여 집합에서 특정 요소를 제거합니다. 요소가 발견되지 않으면 무시합니다.	구문: <pre>set_name.discard(element)</pre> 예제: <pre>fruits.discard("apple")</pre>
issubset()	<code>`issubset()`</code> 메서드는 현재 집합이 다른 집합의 부분집합인지 확인합니다. 현재 집합의 모든 요소가 다른 집합에 존재하면 True를 반환하고, 그렇지 않으면 False를 반환합니다.	구문: <pre>is_subset = set1.issubset(set2)</pre> 예제: <pre>is_subset = fruits.issubset(colors)</pre>
issuperset()	<code>`issuperset()`</code> 메서드는 현재 집합이 다른 집합의 상위 집합인지 확인합니다. 다른 집합의 모든 요소가 현재 집합에 존재하면 True를 반환하고, 그렇지 않으면 False를 반환합니다.	구문: <pre>is_superset = set1.issuperset(set2)</pre> 예제: <pre>is_superset = colors.issuperset(fruits)</pre>
pop()	<code>`pop()`</code> 메서드는 집합에서 임의의 요소를 제거하고 반환합니다. 집합이 비어 있으면 <code>`KeyError`</code> 가 발생합니다. 순서가 중요하지 않을 때 요소를 제거하는 데 이 메서드를 사용합니다.	구문: <pre>removed_element = set_name.pop()</pre>

		예제: <pre>removed_fruit = fruits.pop()</pre>
remove()	<code>`remove()`</code> 메서드를 사용하여 집합에서 특정 요소를 제거합니다. 요소가 발견되지 않으면 <code>`KeyError`</code>가 발생합니다.	구문: <pre>set_name.remove(element)</pre> 예제: <pre>fruits.remove("banana")</pre>
집합 연산	집합에서 다양한 연산을 수행합니다: <code>`합집합`</code>, <code>`교집합`</code>, <code>`차집합`</code>, <code>`대칭 차집합`</code>.	구문: <pre>union_set = set1.union(set2) intersection_set = set1.intersection(set2) difference_set = set1.difference(set2) sym_diff_set = set1.symmetric_difference(set2)</pre> 예제: <pre>combined = fruits.union(colors) common = fruits.intersection(colors) unique_to_fruits = fruits.difference(colors) sym_diff = fruits.symmetric_difference(colors)</pre>
update()	<code>`update()`</code> 메서드는 다른 iterable에서 집합으로 요소를 추가합니다. 요소의 고유성을 유지합니다.	구문: <pre>set_name.update(iterable)</pre> 예제:

```
fruits.update(["kiwi", "grape"])
```



Skills Network

© IBM Corporation. All rights reserved.